

Document de Conception

AGT ERP : Module 1 : RBAC + Tableau de bord squelette

Auteur	Hapket Darelle (cheffe d'équipe)
Encadrant	NOMO BODIANGA Gabriel
Semaine	S1 — 27 au 31 juillet 2026
Version	1.0
Statut	Document de conception — phase S1

Table des matières

1	Introduction	2
2	Architecture du système	2
2.1	Architecture retenue : Client/Serveur	2
2.2	Description de l'architecture	2
2.3	Pourquoi Client/Serveur et non Monolithique ?	3
2.4	Pourquoi pas Microservices ?	3
3	Choix technologiques justifiés	4
4	Diagrammes d'états/transitions	4
4.1	Cycle de vie d'un compte utilisateur (StatutCompte)	4
4.2	Cycle de vie du rôle global (RoleGlobal)	5
5	Diagrammes d'activités	6
5.1	Flux de connexion (login)	6
5.2	Flux d'assignation d'un rôle global	7
5.3	Flux de vérification d'une permission	8
6	Structure des fichiers	8
6.1	Principes d'extensibilité	9
7	Conclusion	9

1. Introduction

Ce document de conception fait suite au document d'analyse du Module 1. Il décrit les décisions techniques retenues pour implémenter le système RBAC à deux niveaux d'AGT ERP, justifie les choix d'architecture et de technologies, et présente les diagrammes de comportement (états/transitions et activités) qui guideront l'implémentation.

2. Architecture du système

2.1 Architecture retenue : Client/Serveur

L'architecture retenue pour AGT ERP est une architecture **client/serveur à deux niveaux** :

- **Client** : application web React (frontend) exécutée dans le navigateur de l'utilisateur.
- **Serveur** : API REST Flask (backend) exposant des endpoints HTTP sécurisés par JWT.
- **Données** : base SQLite persistée dans un volume Docker.

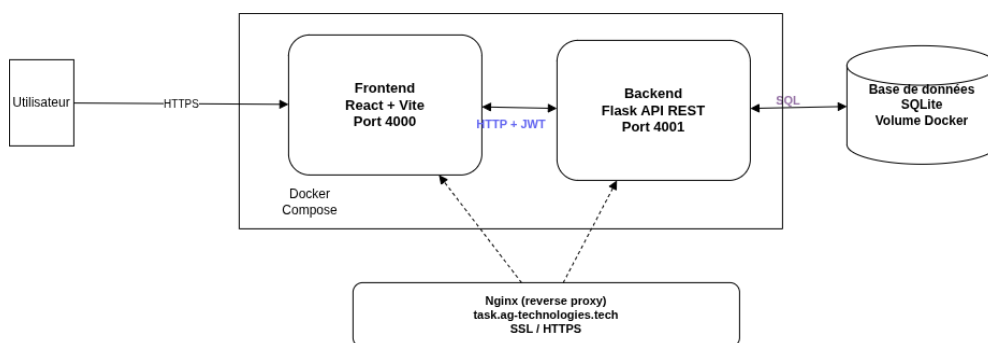


FIGURE 1 – Architecture client/serveur d'AGT ERP

2.2 Description de l'architecture

L'architecture d'AGT ERP est une architecture **client/serveur à deux niveaux**, entièrement conteneurisée via Docker Compose et déployée sur un serveur VPS Hostinger.

Le Frontend est une application React compilée par Vite et servie par un serveur Nginx embarqué dans le conteneur. Il s'exécute dans le navigateur de l'utilisateur et communique avec le backend exclusivement via des requêtes HTTP. Il est accessible sur le port 4000 en développement et via le domaine `task.ag-technologies.tech` en production.

Le Backend est une API REST développée avec Flask (Python). Il expose des endpoints HTTP sécurisés par token JWT. C'est lui qui contient toute la logique métier, vérifie les permissions et accède à la base de données. Il tourne sur le port 4001.

La base de données est SQLite, un fichier unique persisté dans un volume Docker. Elle survit aux redémarrages des conteneurs. Les migrations de schéma sont non-destructives et s'exécutent automatiquement au démarrage via `init_db()`.

Nginx joue le rôle de reverse proxy sur le serveur de production. Il reçoit toutes les requêtes HTTPS sur le domaine `task.ag-technologies.tech`, les redirige vers le frontend (port 4000) ou vers le backend (port 4001) selon le préfixe `/api/`. Le certificat SSL est géré par Certbot (Let's Encrypt) avec renouvellement automatique.

Docker Compose orchestre les deux conteneurs (frontend et backend) en une seule commande : `docker compose up --build -d`. Cela garantit que l'environnement de développement et l'environnement de production sont identiques, éliminant les problèmes de “ça marche sur mon poste”.

Le flux d'une requête typique :

1. L'utilisateur interagit avec l'interface React dans son navigateur.
2. Le frontend envoie une requête HTTP avec le token JWT dans l'en-tête **Authorization**.
3. Le backend reçoit la requête, valide le token, vérifie les permissions via `permissions.py` et traite la logique métier.
4. Le backend interroge la base SQLite et renvoie la réponse en JSON.
5. Le frontend met à jour l'interface selon les données reçues.

2.3 Pourquoi Client/Serveur et non Monolithique ?

Une architecture **monolithique** regroupe interface, logique métier et accès aux données dans un seul et même programme. Elle a été écartée pour les raisons suivantes :

Critère	Justification
Séparation des responsabilités	Le frontend (React) et le backend (Flask) évoluent indépendamment. Un changement d'interface n'oblige pas à toucher la logique métier.
Sécurité	Le backend est la seule source de vérité pour les permissions. Le frontend ne peut pas contourner les contrôles d'accès.
Testabilité	L'API REST peut être testée indépendamment du frontend (avec curl, Postman, tests automatisés).
Réutilisabilité	L'API peut être consommée par d'autres clients (application mobile, scripts d'automatisation) sans modifier le backend.
Travail en équipe	Deux postes travaillent en parallèle (poste A et poste B) sur des périmètres distincts sans se bloquer mutuellement.

2.4 Pourquoi pas Microservices ?

Une architecture microservices (un service indépendant par module : authentification, projets, tâches, etc.) a été écartée car :

- La complexité opérationnelle (orchestration, communication inter-services, monitoring distribué) est disproportionnée pour une équipe de 4 stagiaires.
- L'équipe n'a pas encore les compétences DevOps requises (Kubernetes, service mesh, etc.).
- La base SQLite n'est pas adaptée à un modèle distribué.
- L'objectif est de livrer un ERP fonctionnel en 6 semaines — les microservices rallongeraient considérablement le temps de mise en place.

Conclusion : l'architecture client/serveur offre le meilleur compromis entre séparation des responsabilités, sécurité, productivité d'équipe et complexité maîtrisée.

3. Choix technologiques justifiés

Composant	Technologie	Justification
Backend	Python Flask	Lger, flexible, adapté aux API REST de petite à moyenne envergure. Pas de surcharge de configuration. Ecosystème Python riche (bibliothèques de calcul, génération de rapports). Maîtrisé par l'équipe.
Frontend	React + Vite	Composants réutilisables, rendu rapide, écosystème riche (recharts pour les graphiques, jsPDF pour les exports). Vite remplace Create React App pour des builds plus rapides. Maîtrisé par l'équipe.
Base de données	SQLite	Zéro configuration, portable, embarquée dans le conteneur. Suffisante pour un usage interne avec une équipe réduite. Persiste dans un volume Docker. Migration future vers PostgreSQL possible sans changer le code applicatif.
Authentification	JWT	Stateless : le serveur ne stocke pas les sessions. Contient le rôle et les permissions de l'utilisateur. Compatible REST. Expiration configurable.
Conteneurisation	Docker Compose	Lancement en une commande (<code>docker compose up --build</code>). Isolation des environnements. Portabilité garantie entre les postes de développement et le serveur de production.
Export PDF	jsPDF	Génération côté navigateur, sans appel serveur supplémentaire. Evite les problèmes d'expiration de token JWT lors de la génération.
Hébergement	VPS Hostinger	Serveur dédié, accessible en production sur <code>task.ag-technologies.tech</code> . Nginx comme reverse proxy, Certbot pour le certificat SSL.

4. Diagrammes d'états/transitions

Les diagrammes d'états modélisent le cycle de vie des objets dont l'énumération associée représente des **étapes de vie séquentielles**. Trois objets satisfont ce critère dans le Module 1.

4.1 Cycle de vie d'un compte utilisateur (StatutCompte)

Un compte passe par les états : **EN_ATTENTE** → **ACTIF** ↔ **SUSPENDU** → **SUPPRIMÉ**.

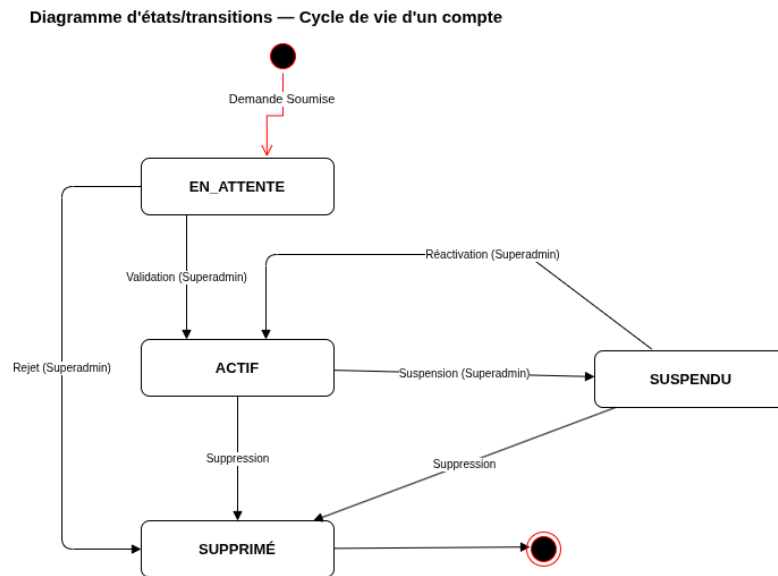


FIGURE 2 – Diagramme d'états — Cycle de vie d'un compte utilisateur

4.2 Cycle de vie du rôle global (RoleGlobal)

Un utilisateur démarre avec le rôle **MEMBRE** et peut être promu **CHEF_PROJET** par le Superadmin, puis rétrogradé.

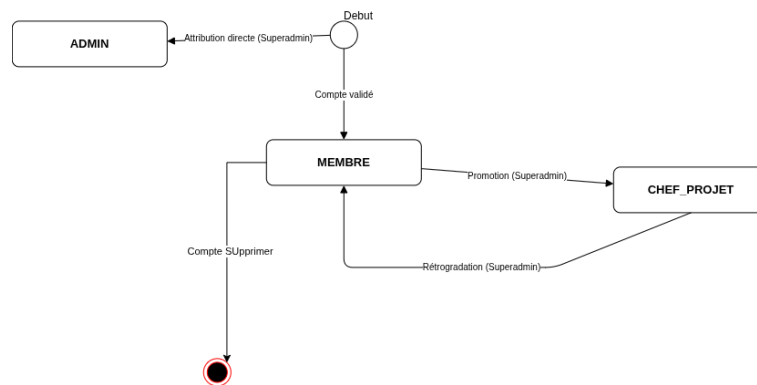


FIGURE 3 – Diagramme d'états — Cycle de vie du rôle global

5. Diagrammes d'activités

5.1 Flux de connexion (login)

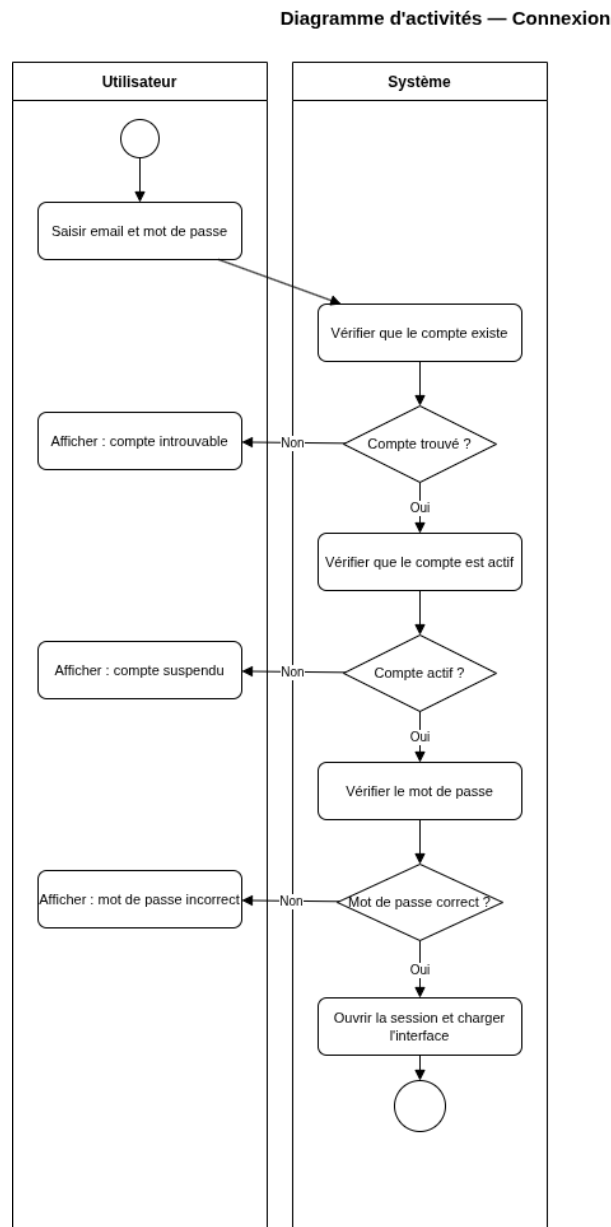


FIGURE 4 – Diagramme d'activités — Connexion

Le flux de connexion se déroule comme suit :

1. L'utilisateur saisit email et mot de passe.
2. Le backend vérifie que le compte existe et que le statut est **ACTIF**.
3. Le mot de passe haché est comparé.
4. Si correct, un token JWT est généré avec le rôle global et les permissions.
5. Le frontend stocke le token et charge l'interface selon les permissions.
6. En cas d'erreur, un message est affiché. En cas de compte suspendu, l'accès est refusé avec un message spécifique.

5.2 Flux d'assignation d'un rôle global

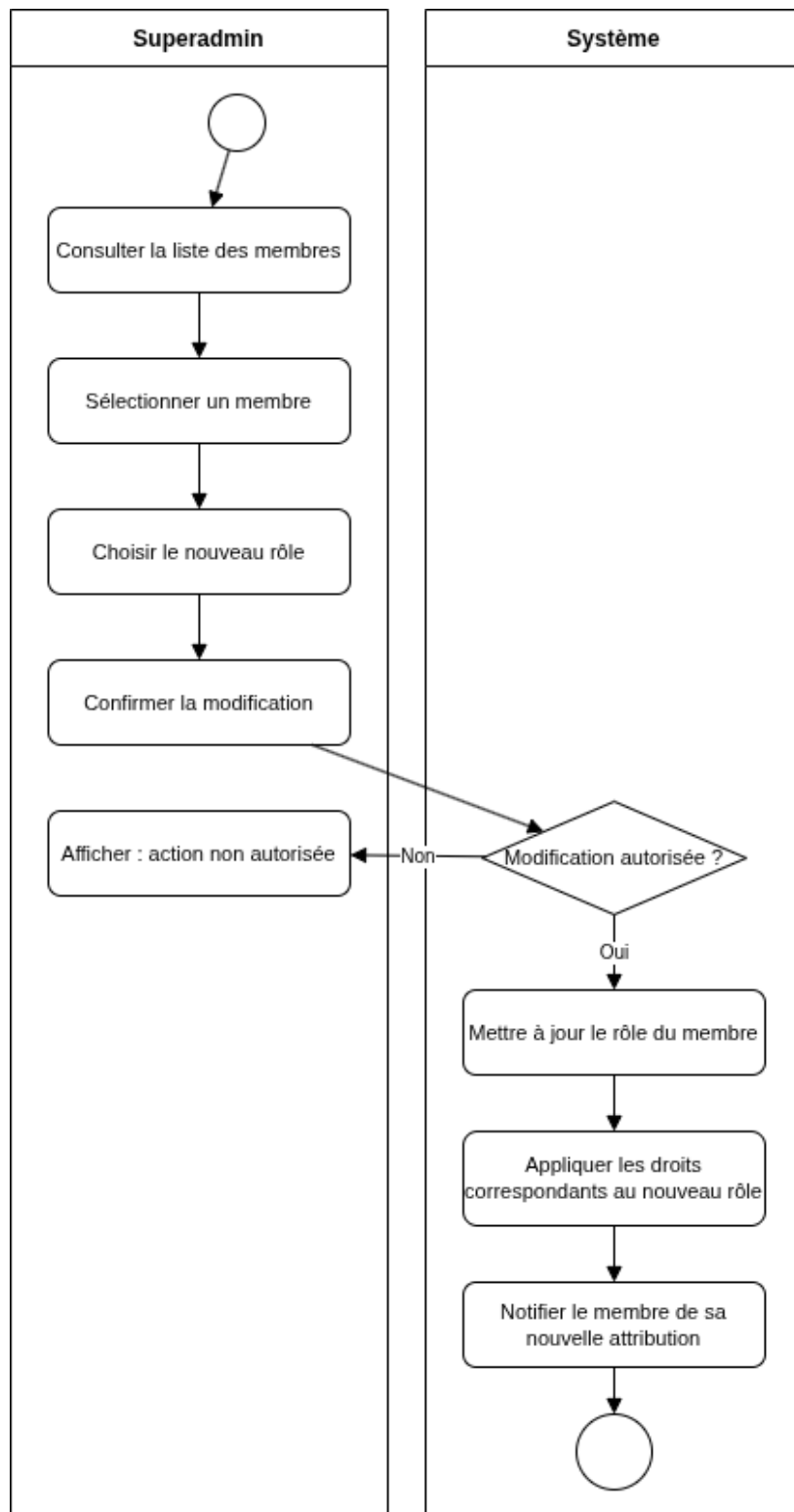


FIGURE 5 – Diagramme d'activités — Assignation d'un rôle global

1. Le Superadmin sélectionne un membre dans la liste de l'équipe.
2. Il choisit le nouveau rôle global.
3. Le backend vérifie que l'acteur est bien Superadmin.

4. Le rôle est mis à jour en base.
5. Les permissions par défaut du nouveau rôle sont copiées dans `PermissionEffective` pour cet utilisateur.
6. Une notification est envoyée au membre.
7. Le membre verra son interface mise à jour au prochain chargement (via `refreshUser()` au montage de l'application).

5.3 Flux de vérification d'une permission

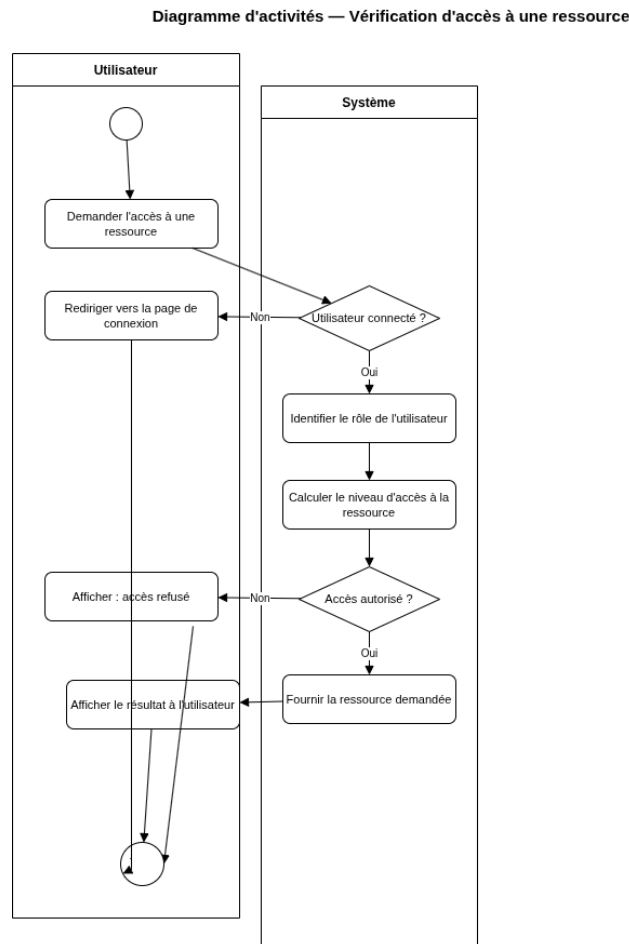


FIGURE 6 – Diagramme d'activités — Vérification d'une permission

1. Une requête HTTP arrive sur un endpoint protégé.
2. Le décorateur `@require_auth` extrait et valide le token JWT.
3. La fonction `get_task_permission_level()` dans `permissions.py` calcule le niveau de permission de l'utilisateur sur la ressource demandée.
4. Si la permission est insuffisante, la réponse `403 Forbidden` est retournée.
5. Sinon, le traitement de la requête se poursuit normalement.

6. Structure des fichiers

La structure des fichiers reflète directement l'architecture client/serveur. Elle est conçue pour être **extensible** : l'ajout d'un nouveau module (Matériel, RH, Finances) ne nécessite que

l'ajout d'un nouveau blueprint côté backend et d'un nouveau composant côté frontend, sans modifier le code existant.

6.1 Principes d'extensibilité

- **Backend** : chaque nouveau module = un nouveau blueprint enregistré dans `app.py`. Les migrations sont ajoutées dans `database.py` sous forme de blocs non-destructifs. Le code existant n'est jamais cassé.
- **Frontend** : chaque nouveau module = un nouveau composant dans `components/`. Il est branché dans `App.jsx` en ajoutant une condition `{tab === "nouveau_module" && <NouveauModule />}`. Les hooks et `client.js` sont partagés — pas de duplication.
- **Permissions** : une nouvelle permission = une ligne dans le seeder de `database.py`. Le système RBAC s'adapte automatiquement sans modifier `permissions.py`.

7. Conclusion

L'architecture client/serveur retenue pour AGT ERP garantit une séparation claire des responsabilités, une sécurité renforcée (le backend reste l'autorité qui fait foi) et une bonne maintenabilité. Les choix technologiques (Flask, React, SQLite, JWT, Docker) sont justifiés par la maîtrise de l'équipe, la rapidité de développement et la portabilité.

Les trois diagrammes d'états/transitions montrent que le cycle de vie des comptes, des rôles globaux et des rôles projet est bien maîtrisé et cohérent avec les règles métier définies dans l'analyse. Les diagrammes d'activités décrivent précisément les flux critiques (connexion, assignation de rôle, vérification de permission) qui seront implémentés en priorité lors de la semaine S1.